



Universidad Nacional Autónoma de México

Facultad de Ciencias

Compiladores. 2025-1 | 7008

Práctica 4:

Definición de un Analizador Sintáctico con BYACC/J

Kevin Steve Quezada Ordoñez

stevequezada@ciencias.unam.mx

21/11/2024



• Dada la gramática $G = (N, \Sigma, P, S)$

Donde P es:

```
programa → declaraciones sentencias
declaraciones → declaraciones declaracion | declaracion
declaracion → tipo lista_var ;
tipo → int | float
lista_var → lista_var , identificador | identificador
sentencias → sentencias sentencia | sentencia
sentencia → identificador = expresion ; | if ( expresion ) sentencias else sentencias
           | while ( expresion ) sentencias
expresion → expresion + expresion | expresion - expresion | expresion * expresion
           | expresion / expresion | identificador | numero
expresion → ( expresion )
```

1. Se determinan los conjuntos N, Σ y el símbolo inicial S . (0.5 pts.)

◦ N (No terminales):

$$N = \{\text{programa}, \text{declaraciones}, \text{declaracion}, \text{tipo}, \text{lista_var}, \text{sentencias}, \text{sentencia}, \text{expresion}\}$$

◦ Σ (Terminales):

$$\Sigma = \{\text{int}, \text{float}, ,, \text{identificador}, =, ;, \text{if}, \text{else}, \text{while}, (,), +, -, *, /, \text{numero}\}$$

◦ S (Símbolo inicial):

$$S = \{\text{programa}\}$$

2. Se realiza el proceso de eliminación de ambigüedad. (1 pts.)

i. Se identifica la ambigüedad

$$\begin{aligned} \text{expresion} \rightarrow & \text{expresion} + \text{expresion} \\ & | \text{expresion} - \text{expresion} \\ & | \text{expresion} * \text{expresion} \\ & | \text{expresion} / \text{expresion} \\ & | \text{identificador} \\ & | \text{numero} \\ & | (\text{expresion}) \end{aligned}$$

Esta gramática es ambigua porque no especifica la precedencia ni la asociatividad de los operadores, permitiendo derivaciones múltiples para una misma expresión. La multiplicación y división tienen mayor precedencia que la suma y resta. Se necesita separar la gramática para reflejar esta diferencia.

ii. Se define la recursividad según la asociatividad

Los operadores +, −, * y / son asociativos a la izquierda. Por lo tanto, las producciones deben ser recursivas a la izquierda para reflejar esta propiedad.

iii. Se reescribe la gramática respetando precedencia y asociatividad

◦ Nivel más bajo:

$$\text{factor} \rightarrow (\text{expresion}) \mid \text{identificador} \mid \text{numero}$$

◦ Nivel intermedio:

$$\text{termino} \rightarrow \text{termino} * \text{factor} \mid \text{termino} / \text{factor} \mid \text{factor}$$

◦ Nivel superior:

$$\text{expresion} \rightarrow \text{expresion} + \text{termino} \mid \text{expresion} - \text{termino} \mid \text{termino}$$

Gramática Actualizada:

$$\begin{aligned} \text{programa} &\rightarrow \text{declaraciones} \text{ sentencias} \\ \text{declaraciones} &\rightarrow \text{declaraciones} \text{ declaracion} \mid \text{declaracion} \\ \text{declaracion} &\rightarrow \text{tipo} \text{ lista_var} ; \\ \text{tipo} &\rightarrow \text{int} \mid \text{float} \\ \text{lista_var} &\rightarrow \text{lista_var}, \text{identificador} \mid \text{identificador} \\ \text{sentencias} &\rightarrow \text{sentencias} \text{ sentencia} \mid \text{sentencia} \\ \text{sentencia} &\rightarrow \text{identificador} = \text{expresion} ; \\ &\mid \text{if} (\text{expresion}) \text{ sentencias} \text{ else} \text{ sentencias} \\ &\mid \text{while} (\text{expresion}) \text{ sentencias} \\ \text{expresion} &\rightarrow \text{expresion} + \text{termino} \mid \text{expresion} - \text{termino} \mid \text{termino} \\ \text{termino} &\rightarrow \text{termino} * \text{factor} \mid \text{termino} / \text{factor} \mid \text{factor} \\ \text{factor} &\rightarrow (\text{expresion}) \mid \text{identificador} \mid \text{numero} \end{aligned}$$

3. Se muestra el proceso de eliminación de la recursividad izquierda. (1 pts.)

i. Se identifica la recursividad izquierda

- $\text{declaraciones} \rightarrow \text{declaraciones declaracion} \mid \text{declaracion}$
- $\text{lista_var} \rightarrow \text{lista_var}, \text{identificador} \mid \text{identificador}$
- $\text{sentencias} \rightarrow \text{sentencias sentencia} \mid \text{sentencia}$
- $\text{expresion} \rightarrow \text{expresion} + \text{termino} \mid \text{expresion} - \text{termino} \mid \text{termino}$
- $\text{termino} \rightarrow \text{termino} * \text{factor} \mid \text{termino} / \text{factor} \mid \text{factor}$

ii. Se elimina la recursividad izquierda.

- **declaraciones:**

$$\begin{aligned} \text{declaraciones} &\rightarrow \text{declaracion declaraciones}' \\ \text{declaraciones}' &\rightarrow \text{declaracion declaraciones}' \mid \epsilon \end{aligned}$$
- **lista_var:**

$$\begin{aligned} \text{lista_var} &\rightarrow \text{identificador lista_var}' \\ \text{lista_var}' &\rightarrow , \text{identificador lista_var}' \mid \epsilon \end{aligned}$$
- **sentencias:**

$$\begin{aligned} \text{sentencias} &\rightarrow \text{sentencia sentencias}' \\ \text{sentencias}' &\rightarrow \text{sentencia sentencias}' \mid \epsilon \end{aligned}$$
- **expresion:**

$$\begin{aligned} \text{expresion} &\rightarrow \text{termino expresion}' \\ \text{expresion}' &\rightarrow + \text{termino expresion}' \\ &\quad \mid - \text{termino expresion}' \mid \epsilon \end{aligned}$$
- **termino:**

$$\begin{aligned} \text{termino} &\rightarrow \text{factor termino}' \\ \text{termino}' &\rightarrow * \text{factor termino}' \\ &\quad \mid / \text{factor termino}' \mid \epsilon \end{aligned}$$

4. Se muestra por qué no es necesario aplicar factorización izquierda. (1 pts.)

La gramática no requiere factorización izquierda debido a que no se presentan producciones con prefijos comunes. Esto significa que las producciones de la gramática comienzan de manera diferente, lo que permite que un parser determine cuál regla aplicar sin ambigüedades.

5. Se muestran los nuevos conjuntos N y P . (0.5 pts.)

- N (No terminales):

$N = \{\text{programa, declaraciones, declaraciones}', \text{declaracion, tipo, lista_var, lista_var}',$
 $\text{sentencias, sentencias}', \text{sentencia, expresion, expresion}', \text{termino, termino}', \text{factor}\}$

- P (Producciones):

```

programa → declaraciones sentencias
declaraciones → declaracion declaraciones'
declaraciones' → declaracion declaraciones' | ε
declaracion → tipo lista_var ;
            tipo → int | float
lista_var → identificador lista_var'
lista_var' → , identificador lista_var' | ε
sentencias → sentencia sentencias'
sentencias' → sentencia sentencias' | ε
sentencia → identificador = expresion ;
           | if ( expresion ) sentencias else sentencias
           | while ( expresion ) sentencias
expresion → termino expresion'
expresion' → + termino expresion'
           | - termino expresion' | ε
termino → factor termino'
termino' → * factor termino'
          | / factor termino' | ε
factor → (expresion)
        | identificador
        | numero

```

● Gramática Modificada.

Para implementar las producciones en **Java**, fue necesario modificar la gramática:

`programa` $\rightarrow$ `declaraciones` `sentencias`

`declaraciones` $\rightarrow$ `declaracion` `declaraciones`'

`declaraciones`' $\rightarrow$ `declaracion` `declaraciones`' $\mid \epsilon$

`declaracion` $\rightarrow$ `tipo` `lista_var` ;

`tipo` $\rightarrow$ `int` $\mid$ `float`

`lista_var` $\rightarrow$ `identificador` `lista_var`'

`lista_var`' $\rightarrow$, `identificador` `lista_var`' $\mid \epsilon$

`sentencias` $\rightarrow$ {`sentencias`'} $\mid$ `sentencia` `sentencias`'

`sentencias`' $\rightarrow$ `sentencia` `sentencias`' $\mid \epsilon$

`sentencia` $\rightarrow$ `identificador` = `expresion` ;

$\mid$ `if` (`expresion`) `sentencias` `else` `sentencias`

$\mid$ `while` (`expresion`) `sentencias`

`expresion` $\rightarrow$ `termino` `expresion`'

`expresion`' $\rightarrow$ + `termino` `expresion`'

$\mid$ - `termino` `expresion`'

$\mid$ < `termino` `expresion`'

$\mid$ > `termino` `expresion`'

$\mid$ <= `termino` `expresion`'

$\mid$ >= `termino` `expresion`'

$\mid$ == `termino` `expresion`'

$\mid$!= `termino` `expresion`'

$\mid \epsilon$

`termino` $\rightarrow$ `factor` `termino`'

`termino`' $\rightarrow$ * `factor` `termino`'

$\mid$ / `factor` `termino`'

$\mid \epsilon$

`factor` $\rightarrow$ (`expresion`)

$\mid$ `identificador`

$\mid$ `numero`

• Eliminación de conflictos shift/reduce:.

1. Estructura General del Programa

- **Original:** programa $\rightarrow$ declaraciones sentencias
- **Nuevo:** programa $\rightarrow$ lista_declaraciones cuerpo_programa EOF
 - Se añadió el Token EOF separado del cuerpo_programa.

2. Cuerpo del Programa

- **Se añadió:**
 - cuerpo_programa $\rightarrow$ bloque_principal | lista_sentencias
 - bloque_principal $\rightarrow$ { lista_sentencias }

3. Declaraciones

- **Original:** declaraciones $\rightarrow$ declaracion declaraciones'
- **Nuevo:** lista_declaraciones $\rightarrow$ lista_declaraciones declaracion | declaracion
 - Se eliminó la regla con epsilon declaraciones'.

4. Lista de Variables

- **Original:** lista_var $\rightarrow$ identificador lista var'
- **Nuevo:** lista_var $\rightarrow$ lista_var , id | id
 - Se eliminó la regla con epsilon lista_var'.

5. Sentencias

- **Original:** sentencias $\rightarrow$ {sentencias'} | sentencia sentencias'
- **Nuevo:**
 - lista_sentencias $\rightarrow$ lista_sentencias sentencia | sentencia
 - sentencias $\rightarrow$ { lista_sentencias } | sentencia

6. Expresiones

- **Original:** Uso de expresion' y termino' con epsilon.
- **Nuevo:** Producciones directas con precedencia:

```
expresion  $\rightarrow$  expresion SUMA termino
          | expresion RESTA termino
          | expresion operador_relacional termino
          | termino
```

7. Términos

- **Original:** `termino → factor termino'`
- **Nuevo:** `termino → termino MULTIPLICACION factor | termino DIVISION factor | factor`

8. Manejo de IF-ELSE

- **Original:** Simple regla if-else.
- **Nuevo:** Se añadió precedencia con `%prec THEN` para resolver el *dangling else*:

```
if_stmt → IF LPAR expresion RPAR sentencias %prec THEN  
         | IF LPAR expresion RPAR sentencias ELSE sentencias
```

9. Factores

- **Original:** `factor → (expresion) | identificador | numero`
- **Nuevo:** `factor → LPAR expresion RPAR | ID | NUMERO_ENTERO | NUMERO_REAL`

10. Precedencia de Operadores

- **Se añadió:** Declaraciones explícitas de precedencia y asociatividad:

```
%left SUMA RESTA  
%left MULTIPLICACION DIVISION  
%nonassoc MENORQUE MAYORQUE MENORIGUAL MAYORIGUAL IGUALDAD DIFERENTE
```

• Gramática Final.

```
programa → lista_declaraciones cuerpo_programa EOF
cuerpo_programa → bloque_principal | lista_sentencias
bloque_principal → {lista_sentencias}

lista_declaraciones → lista_declaraciones declaracion | declaracion
declaracion → tipo lista_var ;
tipo → int | float
lista_var → lista_var , id | id

lista_sentencias → lista_sentencias sentencia | sentencia
sentencias → {lista_sentencias} | sentencia

sentencia → asignacion
           | if_stmt
           | while_stmt

asignacion → id = expresion ;

if_stmt → if (expresion) sentencias
        | if (expresion) sentencias else sentencias

while_stmt → while (expresion) sentencias

expresion → expresion + termino
          | expresion - termino
          | expresion < termino
          | expresion > termino
          | expresion <= termino
          | expresion >= termino
          | expresion == termino
          | expresion != termino
          | termino

termino → termino * factor
        | termino / factor
        | factor

factor → (expresion)
       | id | numero_entero | numero_real
```